

Bootstrap Components

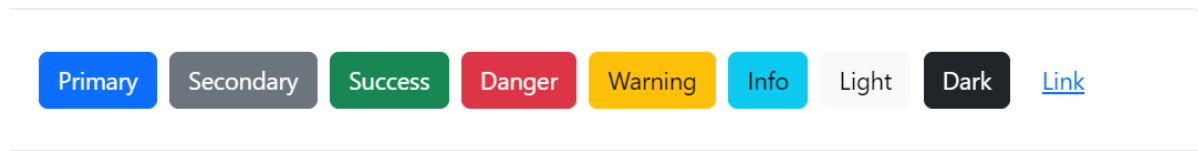
Bootstrap is a popular front-end framework used for designing and building responsive websites and web applications. Here are some key points about Bootstrap in CSS:

1. **Bootstrap – button:** Use Bootstrap's custom button styles for actions in forms, dialogs, and more with support for multiple sizes, states, and more.

Bootstrap has a base `.btn` class that sets up basic styles such as padding and content alignment. By default, `.btn` controls have a transparent border and background color, and lack any explicit focus and hover styles.

Code :

```
<button type="button" class="btn btn-primary">Primary</button>
<button type="button" class="btn btn-secondary">Secondary</button>
<button type="button" class="btn btn-success">Success</button>
<button type="button" class="btn btn-danger">Danger</button>
<button type="button" class="btn btn-warning">Warning</button>
<button type="button" class="btn btn-info">Info</button>
<button type="button" class="btn btn-light">Light</button>
<button type="button" class="btn btn-dark">Dark</button>
```



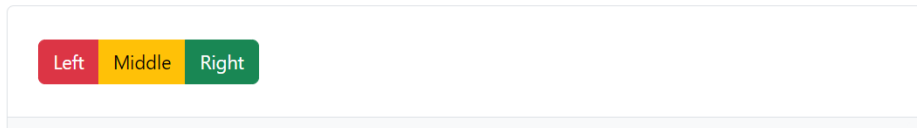
Bootstrap Website Link : <https://getbootstrap.com/docs/5.3/components/buttons/#base-class>

2. **Bootstrap -button group:** Group a series of buttons together on a single line or stack them in a vertical column. Wrap a series of buttons with `.btn` in `.btn-group`.

Code :

```
<div class="btn-group" role="group" aria-label="Basic mixed styles example">
  <button type="button" class="btn btn-danger">Left</button>
  <button type="button" class="btn btn-warning">Middle</button>
  <button type="button" class="btn btn-success">Right</button>
</div>
```

Mixed styles



Checkout the official website link : <https://getbootstrap.com/docs/5.3/components/button-group/#basic-example>

3. **Bootstrap - Input group** : Easily extend form controls by adding text, buttons, or button groups on either side of textual inputs, custom selects, and custom file inputs.

Place one add-on or button on either side of an input. You may also place one on both sides of an input. Remember to place <label>s outside the input group.

Code :

```
<div class="input-group mb-3">
  <span class="input-group-text" id="basic-addon1">@</span>
  <input type="text" class="form-control" placeholder="Username" aria-label="Username" aria-describedby="basic-addon1">
</div>

<div class="input-group mb-3">
  <input type="text" class="form-control" placeholder="Recipient's username" aria-label="Recipient's username" aria-describedby="basic-addon2">
  <span class="input-group-text" id="basic-addon2">@example.com</span>
</div>

<div class="mb-3">
  <label for="basic-url" class="form-label">Your vanity URL</label>
  <div class="input-group">
    <span class="input-group-text" id="basic-addon3">https://example.com/users/</span>
    <input type="text" class="form-control" id="basic-url" aria-describedby="basic-addon3 basic-addon4">
  </div>
  <div class="form-text" id="basic-addon4">Example help text goes outside the input group.</div>
</div>
```

```
<div class="input-group mb-3">
  <span class="input-group-text">$</span>
  <input type="text" class="form-control" aria-label="Amount (to the
nearest dollar)">
  <span class="input-group-text">.00</span>
</div>

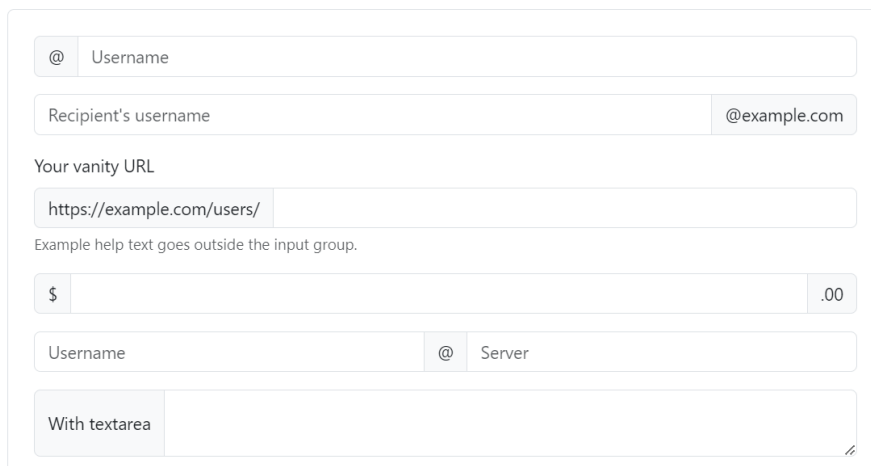
<div class="input-group mb-3">
  <input type="text" class="form-control" placeholder="Username" aria-
label="Username">
  <span class="input-group-text">@</span>
  <input type="text" class="form-control" placeholder="Server" aria-
label="Server">
</div>

<div class="input-group">
  <span class="input-group-text">With textarea</span>
  <textarea class="form-control" aria-label="With textarea"></textarea>
</div>
```

Result:

Basic example |

Place one add-on or button on either side of an input. You may also place one on both sides of an input. Remember to place `<label>`s outside the input group.



Checkout the official website link : <https://getbootstrap.com/docs/5.3/components/button-group/#basic-example>

4. Bootstrap -Floating labels:

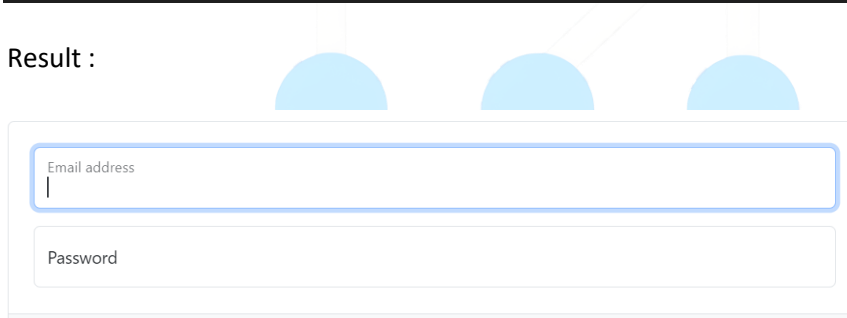
Create beautifully simple form labels that float over your input fields.

Wrap a pair of `<input class="form-control">` and `<label>` elements in `.form-floating` to enable floating labels with Bootstrap's textual form fields. A placeholder is required on each `<input>` as our method of CSS-only floating labels uses the `:placeholder-shown` pseudo-element. Also note that the `<input>` must come first so we can utilize a sibling selector (e.g., `~`).

Code :

```
<div class="form-floating mb-3">
  <input type="email" class="form-control" id="floatingInput"
placeholder="name@example.com">
  <label for="floatingInput">Email address</label>
</div>
<div class="form-floating">
  <input type="password" class="form-control" id="floatingPassword"
placeholder="Password">
  <label for="floatingPassword">Password</label>
</div>
```

Result :



Checkout the official website link : <https://getbootstrap.com/docs/5.3/forms/floating-labels/#example>

5. Bootstrap Forms: Here are some Examples and usage guidelines for form control styles, layout options, and custom components for creating a wide variety of forms.

1. **Form control** : Style textual inputs and textareas with support for multiple states.
2. **Select** : Improve browser default select elements with a custom initial appearance.
3. **Checks & radios** : Use our custom radio buttons and checkboxes in forms for selecting input options.
4. **Range** : Replace browser default range inputs with our custom version.
5. **Input group**: Attach labels and buttons to your inputs for increased semantic value.
6. **Floating labels**: Create beautifully simple form labels that float over your input fields.
7. **Layout**: Create inline, horizontal, or complex grid-based layouts with your forms.
8. **Validation**: Validate your forms with custom or native validation behaviors and styles.

Bootstrap's form controls expand on [our Rebooted form styles](#) with classes. Use these classes to opt into their customized displays for a more consistent rendering across browsers and devices.

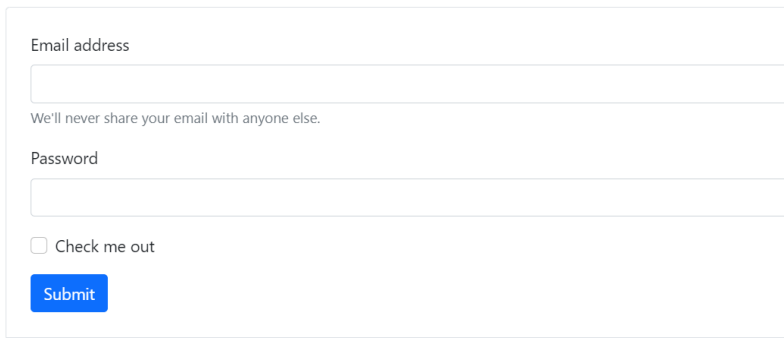
Be sure to use an appropriate **type** attribute on all inputs (e.g., **email** for email address or **number** for numerical information) to take advantage of newer input controls like email verification, number selection, and more.

Here's a quick example to demonstrate Bootstrap's form styles. Keep reading for documentation on required classes, form layout, and more.

Code :

```
<form>
  <div class="mb-3">
    <label for="exampleInputEmail1" class="form-label">Email
address</label>
    <input type="email" class="form-control" id="exampleInputEmail1"
aria-describedby="emailHelp">
    <div id="emailHelp" class="form-text">We'll never share your email
with anyone else.</div>
  </div>
  <div class="mb-3">
    <label for="exampleInputPassword1" class="form-
label">Password</label>
    <input type="password" class="form-control"
id="exampleInputPassword1">
  </div>
  <div class="mb-3 form-check">
    <input type="checkbox" class="form-check-input" id="exampleCheck1">
    <label class="form-check-label" for="exampleCheck1">Check me
out</label>
  </div>
  <button type="submit" class="btn btn-primary">Submit</button>
</form>
```

Result:



Email address

We'll never share your email with anyone else.

Password

☐ Check me out

Submit

Checkout the official website link : <https://getbootstrap.com/docs/5.1/forms/overview/>

6. **Bootstrap Dropwons** : Dropdowns are toggleable, contextual overlays for displaying lists of links and more. They're made interactive with the included Bootstrap dropdown JavaScript plugin. They're toggled by clicking, not by hovering; this is an intentional design decision.

Dropdowns are built on a third party library, Popper, which provides dynamic positioning and viewport detection. Be sure to include **popper.min.js** before Bootstrap's JavaScript or use **bootstrap.bundle.min.js** / **bootstrap.bundle.js** which contains Popper. Popper isn't used to position dropdowns in navbars though as dynamic positioning isn't required.

Wrap the dropdown's toggle (your button or link) and the dropdown menu within `.dropdown`, or another element that declares `position: relative;`. Ideally, you should use a **<button>** element as the dropdown trigger, but the plugin will work with `<a>` elements as well. The examples shown here use semantic `<ul>` elements where appropriate, but custom markup is supported.

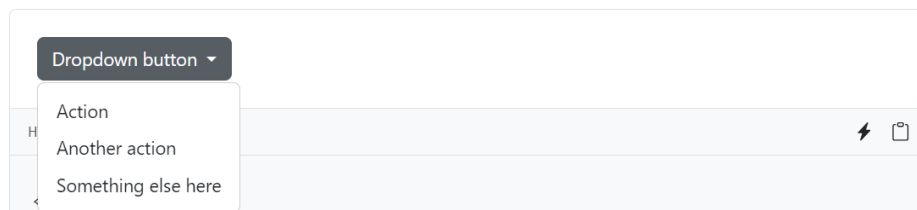
Single button

Any single `.btn` can be turned into a dropdown toggle with some markup changes. Here's how you can put them to work with `<button>` elements:

Example Snippet :

```
<div class="dropdown">
  <button class="btn btn-secondary dropdown-toggle" type="button" data-
bs-toggle="dropdown" aria-expanded="false">
    Dropdown button
  </button>
  <ul class="dropdown-menu">
    <li><a class="dropdown-item" href="#">Action</a></li>
    <li><a class="dropdown-item" href="#">Another action</a></li>
    <li><a class="dropdown-item" href="#">Something else here</a></li>
  </ul>
</div>
```

Result :



Checkout the official website link :

<https://getbootstrap.com/docs/5.3/components/dropdowns/#overview>

7. Bootstrap checkbox and radio buttons:

Create consistent cross-browser and cross-device checkboxes and radios with our completely rewritten checks component.

Browser default checkboxes and radios are replaced with the help of **.form-check**, a series of classes for both input types that improves the layout and behavior of their HTML elements, that provide greater customization and cross browser consistency. Checkboxes are for selecting one or several options in a list, while radios are for selecting one option from many.

Structurally, our **<input>**s and **<label>**s are sibling elements as opposed to an **<input>** within a **<label>**. This is slightly more verbose as you must specify id and for attributes to relate the **<input>** and **<label>**. We use the sibling selector (**~**) for all our **<input>** states, like **:checked** or **:disabled**. When combined with the **.form-check-label** class, we can easily style the text for each item based on the **<input>**'s state.

Code Checkbox :

```
<div class="form-check">
  <input class="form-check-input" type="checkbox" value=""
id="flexCheckDefault">
  <label class="form-check-label" for="flexCheckDefault">
    Default checkbox
  </label>
</div>
<div class="form-check">
  <input class="form-check-input" type="checkbox" value=""
id="flexCheckChecked" checked>
  <label class="form-check-label" for="flexCheckChecked">
    Checked checkbox
  </label>
</div>
```

Result :

- ☐ Default checkbox
☒ Checked checkbox

Code Radio:

```
<div class="form-check">
  <input class="form-check-input" type="radio" name="flexRadioDefault"
id="flexRadioDefault1">
  <label class="form-check-label" for="flexRadioDefault1">
    Default radio
  </label>
</div>
<div class="form-check">
  <input class="form-check-input" type="radio" name="flexRadioDefault"
id="flexRadioDefault2" checked>
  <label class="form-check-label" for="flexRadioDefault2">
    Default checked radio
  </label>
</div>
```

Result :

- ☐ Default radio
☒ Default checked radio

Checkout the official website link : <https://getbootstrap.com/docs/5.3/forms/checks-radios/>

- Bootstrap cards** : Bootstrap's cards provide a flexible and extensible content container with multiple variants and options..

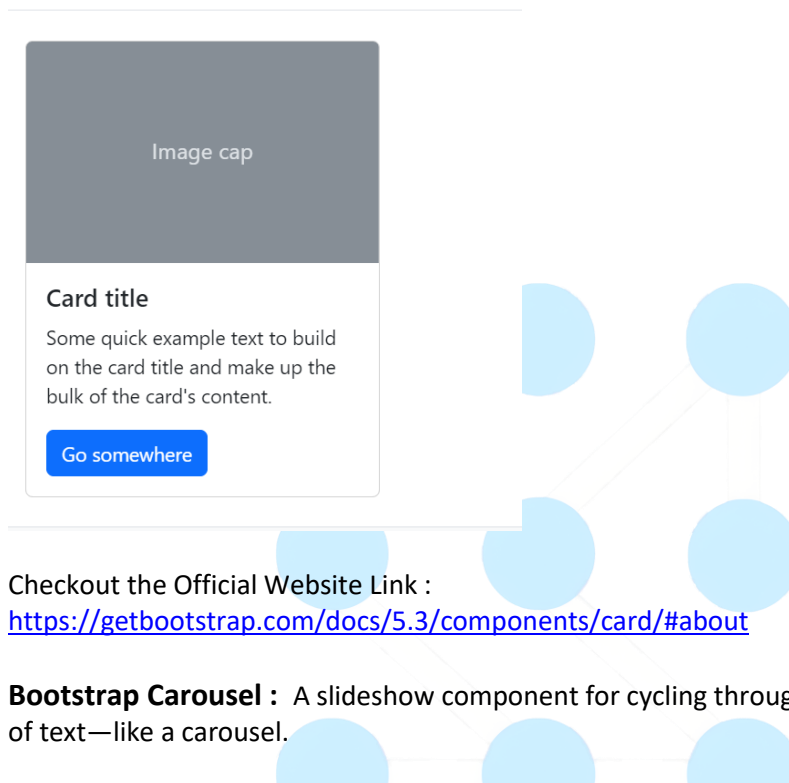
A **card** is a flexible and extensible content container. It includes options for headers and footers, a wide variety of content, contextual background colors, and powerful display options. If you're familiar with Bootstrap 3, cards replace our old panels, wells, and thumbnails. Similar functionality to those components is available as modifier classes for cards.

Code Cards :

```
<div class="card" style="width: 18rem;">
  
  <div class="card-body">
    <h5 class="card-title">Card title</h5>
```

```
<p class="card-text">Some quick example text to build on the card
title and make up the bulk of the card's content.</p>
<a href="#" class="btn btn-primary">Go somewhere</a>
</div>
</div>
```

Result :



Checkout the Official Website Link :

<https://getbootstrap.com/docs/5.3/components/card/#about>

9. **Bootstrap Carousel** : A slideshow component for cycling through elements—images or slides of text—like a carousel.

How it works

- The carousel is a slideshow for cycling through a series of content, built with CSS 3D transforms and a bit of JavaScript. It works with a series of images, text, or custom markup. It also includes support for previous/next controls and indicators.
- For performance reasons, **carousels must be manually initialized** using the [carousel constructor method](#). Without initialization, some of the event listeners (specifically, the events needed touch/swipe support) will not be registered until a user has explicitly activated a control or indicator.

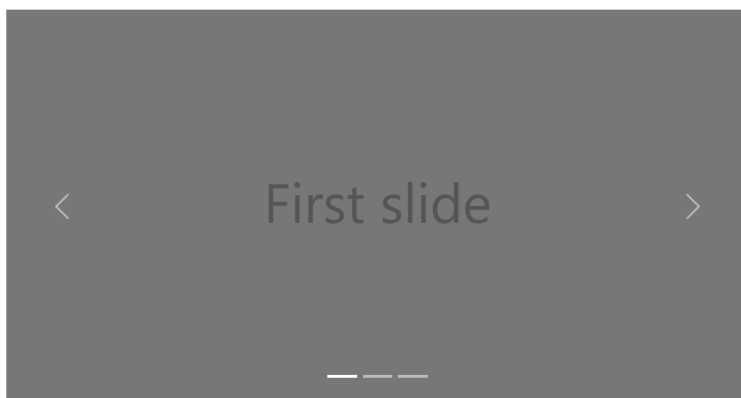
The only exception are [autoplaying carousels](#) with the data-bs-ride="carousel" attribute as these are initialized automatically on page load. If you're using autoplaying carousels with the data attribute, **don't explicitly initialize the same carousels with the constructor method.**

- Nested carousels are not supported. You should also be aware that carousels in general can often cause usability and accessibility challenges.

Code :

```
<div id="carouselExampleIndicators" class="carousel slide">
  <div class="carousel-indicators">
    <button type="button" data-bs-target="#carouselExampleIndicators"
data-bs-slide-to="0" class="active" aria-current="true" aria-label="Slide
1"></button>
    <button type="button" data-bs-target="#carouselExampleIndicators"
data-bs-slide-to="1" aria-label="Slide 2"></button>
    <button type="button" data-bs-target="#carouselExampleIndicators"
data-bs-slide-to="2" aria-label="Slide 3"></button>
  </div>
  <div class="carousel-inner">
    <div class="carousel-item active">
      
    </div>
    <div class="carousel-item">
      
    </div>
    <div class="carousel-item">
      
    </div>
  </div>
  <button class="carousel-control-prev" type="button" data-bs-
target="#carouselExampleIndicators" data-bs-slide="prev">
    <span class="carousel-control-prev-icon" aria-hidden="true"></span>
    <span class="visually-hidden">Previous</span>
  </button>
  <button class="carousel-control-next" type="button" data-bs-
target="#carouselExampleIndicators" data-bs-slide="next">
    <span class="carousel-control-next-icon" aria-hidden="true"></span>
    <span class="visually-hidden">Next</span>
  </button>
</div>
```

Result :



Checkout the official website link here :

<https://getbootstrap.com/docs/5.3/components/carousel/#how-it-works>

10. **Bootstrap Collapse:** Toggle the visibility of content across your project with a few classes and our JavaScript plugins.

How It Works :

The collapse JavaScript plugin is used to show and hide content. Buttons or anchors are used as triggers that are mapped to specific elements you toggle. Collapsing an element will animate the height from its current value to 0. Given how CSS handles animations, you cannot use padding on a `.collapse` element. Instead, use the class as an independent wrapping element.

Click the buttons below to show and hide another element via class changes:

- `.collapse` hides content
- `.collapsing` is applied during transitions
- `.collapse.show` shows content

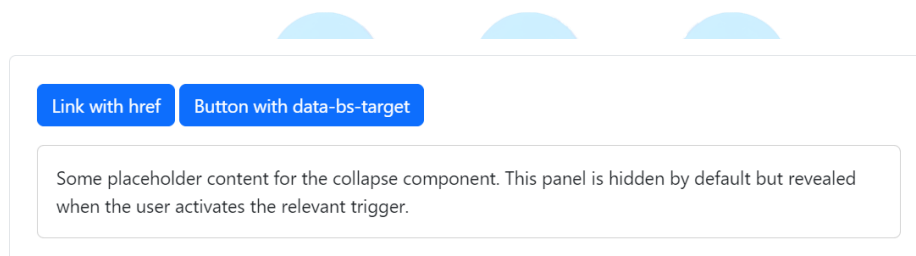
Generally, we recommend using a `<button>` with the `data-bs-target` attribute. While not recommended from a semantic point of view, you can also use an `<a>` link with the `href` attribute (and a `role="button"`). In both cases, the `data-bs-toggle="collapse"` is required.

Code :

```
<p class="d-inline-flex gap-1">
  <a class="btn btn-primary" data-bs-toggle="collapse"
href="#collapseExample" role="button" aria-expanded="false" aria-
controls="collapseExample">
    Link with href
  </a>
```

```
<button class="btn btn-primary" type="button" data-bs-
toggle="collapse" data-bs-target="#collapseExample" aria-expanded="false"
aria-controls="collapseExample">
  Button with data-bs-target
</button>
</p>
<div class="collapse" id="collapseExample">
  <div class="card card-body">
    Some placeholder content for the collapse component. This panel is
    hidden by default but revealed when the user activates the relevant
    trigger.
  </div>
</div>
```

Result :



Checkout the official website link here :

<https://getbootstrap.com/docs/5.3/components/collapse/#how-it-works>

11. **Bootstrap Accordion:** Build vertically collapsing accordions in combination with our Collapse JavaScript plugin.

Example

Click the accordions below to expand/collapse the accordion content.

To render an accordion that's expanded by default:

- add the **.show** class on the **.accordion-collapse** element.
- drop the **.collapsed** class from the **.accordion-button** element and set its **aria-expanded** attribute to true.

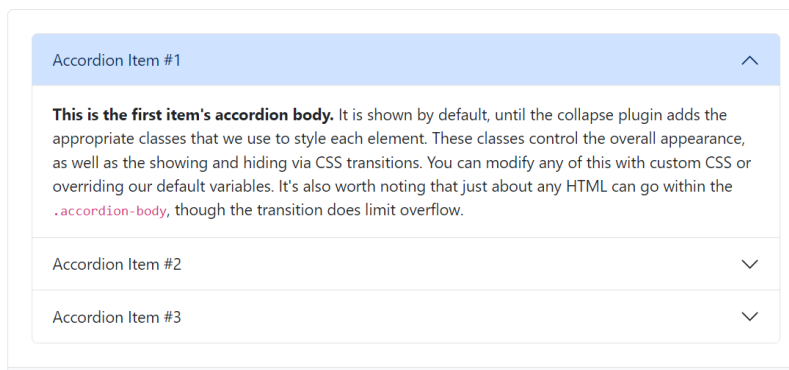
Code :

```
<div class="accordion" id="accordionExample">
  <div class="accordion-item">
    <h2 class="accordion-header">
```

```
<button class="accordion-button" type="button" data-bs-
toggle="collapse" data-bs-target="#collapseOne" aria-expanded="true" aria-
controls="collapseOne">
    Accordion Item #1
</button>
</h2>
<div id="collapseOne" class="accordion-collapse collapse show" data-
bs-parent="#accordionExample">
    <div class="accordion-body">
        <strong>This is the first item's accordion body.</strong> It is
shown by default, until the collapse plugin adds the appropriate classes
that we use to style each element. These classes control the overall
appearance, as well as the showing and hiding via CSS transitions. You can
modify any of this with custom CSS or overriding our default variables.
It's also worth noting that just about any HTML can go within the
<code>.accordion-body</code>, though the transition does limit overflow.
    </div>
</div>
<div class="accordion-item">
    <h2 class="accordion-header">
        <button class="accordion-button collapsed" type="button" data-bs-
toggle="collapse" data-bs-target="#collapseTwo" aria-expanded="false"
aria-controls="collapseTwo">
            Accordion Item #2
        </button>
    </h2>
    <div id="collapseTwo" class="accordion-collapse collapse" data-bs-
parent="#accordionExample">
        <div class="accordion-body">
            <strong>This is the second item's accordion body.</strong> It is
hidden by default, until the collapse plugin adds the appropriate classes
that we use to style each element. These classes control the overall
appearance, as well as the showing and hiding via CSS transitions. You can
modify any of this with custom CSS or overriding our default variables.
It's also worth noting that just about any HTML can go within the
<code>.accordion-body</code>, though the transition does limit overflow.
        </div>
    </div>
</div>
<div class="accordion-item">
    <h2 class="accordion-header">
```

```
<button class="accordion-button collapsed" type="button" data-bs-
toggle="collapse" data-bs-target="#collapseThree" aria-expanded="false"
aria-controls="collapseThree">
    Accordion Item #3
</button>
</h2>
<div id="collapseThree" class="accordion-collapse collapse" data-bs-
parent="#accordionExample">
    <div class="accordion-body">
        <strong>This is the third item's accordion body.</strong> It is
hidden by default, until the collapse plugin adds the appropriate classes
that we use to style each element. These classes control the overall
appearance, as well as the showing and hiding via CSS transitions. You can
modify any of this with custom CSS or overriding our default variables.
It's also worth noting that just about any HTML can go within the
<code>.accordion-body</code>, though the transition does limit overflow.
    </div>
</div>
</div>
</div>
```

Result :



Checkout the official website link here :

<https://getbootstrap.com/docs/5.3/components/accordion/#how-it-works>

12. **Bootstrap Alerts** : Provide contextual feedback messages for typical user actions with the handful of available and flexible alert messages.

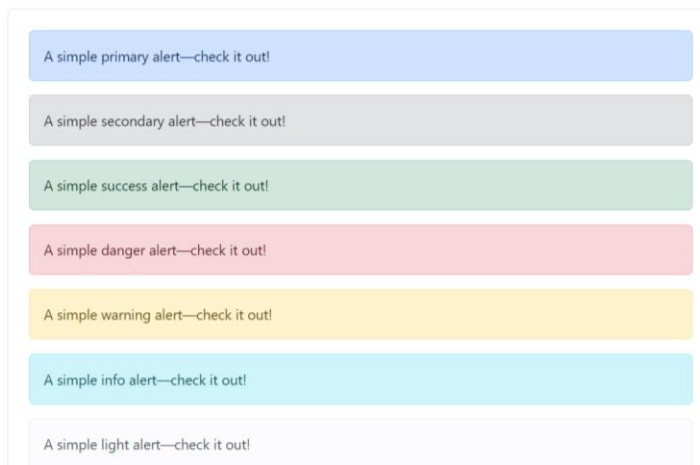
How it works :

Alerts are available for any length of text, as well as an optional close button. For proper styling, use one of the eight **required** contextual classes (e.g., `.alert-success`). For inline dismissal, use the [alerts JavaScript plugin](#)

Code :

```
<div class="alert alert-primary" role="alert">
  A simple primary alert—check it out!
</div>
<div class="alert alert-secondary" role="alert">
  A simple secondary alert—check it out!
</div>
<div class="alert alert-success" role="alert">
  A simple success alert—check it out!
</div>
<div class="alert alert-danger" role="alert">
  A simple danger alert—check it out!
</div>
<div class="alert alert-warning" role="alert">
  A simple warning alert—check it out!
</div>
<div class="alert alert-info" role="alert">
  A simple info alert—check it out!
</div>
<div class="alert alert-light" role="alert">
  A simple light alert—check it out!
</div>
<div class="alert alert-dark" role="alert">
  A simple dark alert—check it out!
</div>
```

Result :



Checkout the official website link here :

<https://getbootstrap.com/docs/5.3/components/alerts/#examples>

13. **Bootstrap Badge** : Documentation and examples for badges, our small count and labeling component.
- Badges scale to match the size of the immediate parent element by using relative font sizing and em units. As of v5, badges no longer have focus or hover styles for links.

Code :

```
<button type="button" class="btn btn-primary position-relative">
  Inbox
  <span class="position-absolute top-0 start-100 translate-middle badge
rounded-pill bg-danger">
    99+
    <span class="visually-hidden">unread messages</span>
  </span>
</button>
```

Result :



14. **Bootstrap Pagination** :

Documentation and examples for showing pagination to indicate a series of related content exists across multiple pages.

We use a large block of connected links for our pagination, making links hard to miss and easily scalable—all while providing large hit areas. Pagination is built with list HTML elements so screen readers can announce the number of available links. Use a wrapping `<nav>` element to identify it as a navigation section to screen readers and other assistive technologies.

In addition, as pages likely have more than one such navigation section, it's advisable to provide a descriptive `aria-label` for the `<nav>` to reflect its purpose. For example, if the pagination component is used to navigate between a set of search results, an appropriate label could be `aria-label="Search results pages"`.

Code :

```
<nav aria-label="...">
  <ul class="pagination">
    <li class="page-item disabled">
      <a class="page-link">Previous</a>
    </li>
    <li class="page-item"><a class="page-link" href="#">1</a></li>
    <li class="page-item active" aria-current="page">
      <a class="page-link" href="#">2</a>
    </li>
    <li class="page-item"><a class="page-link" href="#">3</a></li>
    <li class="page-item">
      <a class="page-link" href="#">Next</a>
    </li>
  </ul>
</nav>
```

Result :



Checkout the Official Website Link :

<https://getbootstrap.com/docs/5.3/components/pagination/#overview>

15. **Bootstrap Placeholders** : Use loading placeholders for your components or pages to indicate something may still be loading.

Placeholders can be used to enhance the experience of your application. They're built only with HTML and CSS, meaning you don't need any JavaScript to create them. You will, however, need some custom JavaScript to toggle their visibility. Their appearance, color, and sizing can be easily customized with our utility classes.

Example

In the example below, we take a typical card component and recreate it with placeholders applied to create a “loading card”. Size and proportions are the same between the two.

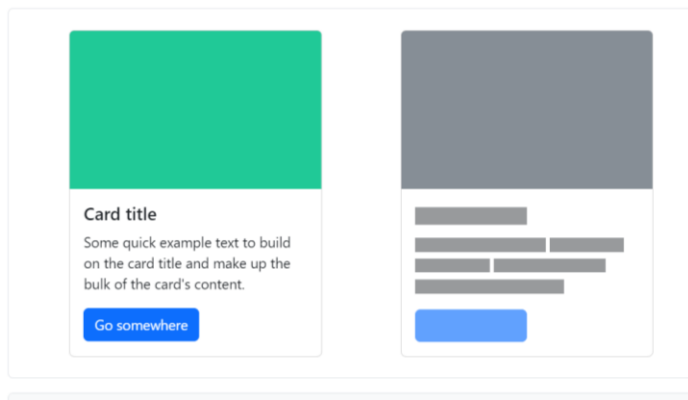
Code :

```
<div class="card">
  

  <div class="card-body">
    <h5 class="card-title">Card title</h5>
    <p class="card-text">Some quick example text to build on the card
title and make up the bulk of the card's content.</p>
    <a href="#" class="btn btn-primary">Go somewhere</a>
  </div>
</div>

<div class="card" aria-hidden="true">
  
  <div class="card-body">
    <h5 class="card-title placeholder-glow">
      <span class="placeholder col-6"></span>
    </h5>
    <p class="card-text placeholder-glow">
      <span class="placeholder col-7"></span>
      <span class="placeholder col-4"></span>
      <span class="placeholder col-4"></span>
      <span class="placeholder col-6"></span>
      <span class="placeholder col-8"></span>
    </p>
    <a class="btn btn-primary disabled placeholder col-6" aria-
disabled="true"></a>
  </div>
</div>
```

Result :



Checkout the official website link :

<https://getbootstrap.com/docs/5.3/components/placeholders/#about>

16. Bootstrap Popovers :

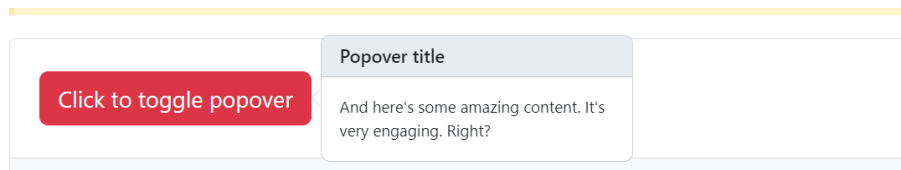
Things to know when using the popover plugin:

- Popovers rely on the third party library [Popper](#) for positioning. You must include [popper.min.js](#) before bootstrap.js, or use one bootstrap.bundle.min.js which contains Popper.
- Popovers require the [popover plugin](#) as a dependency.
- Popovers are opt-in for performance reasons, so **you must initialize them yourself**.
- Zero-length title and content values will never show a popover.
- Specify container: 'body' to avoid rendering problems in more complex components (like our input groups, button groups, etc).
- Triggering popovers on hidden elements will not work.
- Popovers for .disabled or disabled elements must be triggered on a wrapper element.
- When triggered from anchors that wrap across multiple lines, popovers will be centered between the anchors' overall width. Use .text-nowrap on your <a>s to avoid this behavior.
- Popovers must be hidden before their corresponding elements have been removed from the DOM.
- Popovers can be triggered thanks to an element inside a shadow DOM.

Code :

```
<button type="button" class="btn btn-lg btn-danger" data-bs-toggle="popover" data-bs-title="Popover title" data-bs-content="And here's some amazing content. It's very engaging. Right?">Click to toggle popover</button>
```

Result :



Checkout the official website with the link :

<https://getbootstrap.com/docs/5.3/components/popovers/#overview>

17. Bootstrap Progress :

How it works

Progress components are built with two HTML elements, some CSS to set the width, and a few attributes. We don't use [the HTML5 <progress> element](#), ensuring you can stack progress bars, animate them, and place text labels over them.

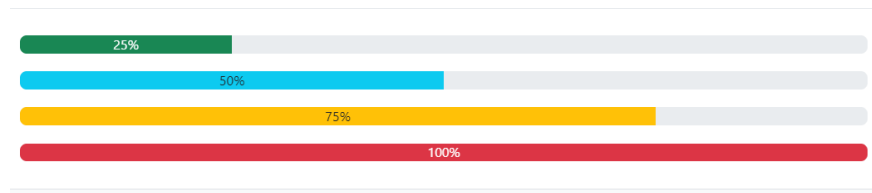
- We use the `.progress` as a wrapper to indicate the max value of the progress bar.
- The `.progress` wrapper also requires a `role="progressbar"` and `aria` attributes to make it accessible, including an accessible name (using `aria-label`, `aria-labelledby`, or similar).
- We use the inner `.progress-bar` purely for the visual bar and label.
- The `.progress-bar` requires an inline style, utility class, or custom CSS to set its width.
- We provide a special `.progress-stacked` class to create multiple/stacked progress bars.

Put that all together, and you have the following examples.

Code :

```
<div class="progress" role="progressbar" aria-label="Success example"
aria-valuenow="25" aria-valuemin="0" aria-valuemax="100">
  <div class="progress-bar bg-success" style="width: 25%">25%</div>
</div>
<div class="progress" role="progressbar" aria-label="Info example" aria-
valuenow="50" aria-valuemin="0" aria-valuemax="100">
  <div class="progress-bar bg-info text-dark" style="width: 50%">50%</div>
</div>
<div class="progress" role="progressbar" aria-label="Warning example"
aria-valuenow="75" aria-valuemin="0" aria-valuemax="100">
  <div class="progress-bar bg-warning text-dark" style="width:
75%">75%</div>
</div>
<div class="progress" role="progressbar" aria-label="Danger example" aria-
valuenow="100" aria-valuemin="0" aria-valuemax="100">
  <div class="progress-bar bg-danger" style="width: 100%">100%</div>
</div>
```

Result:



Checkout the official website with the link :

<https://getbootstrap.com/docs/5.3/components/progress/#how-it-works>

18. Bootstrap Spinners: Indicate the loading state of a component or page with Bootstrap spinners, built entirely with HTML, CSS, and no JavaScript.

#About:

Bootstrap “spinners” can be used to show the loading state in your projects. They’re built only with HTML and CSS, meaning you don’t need any JavaScript to create them. You will, however, need some custom JavaScript to toggle their visibility. Their appearance, alignment, and sizing can be easily customized with our amazing utility classes.

For accessibility purposes, each loader here includes `role="status"` and a nested `<span class="visually-hidden">Loading...</span>`.

Code :

```
<div class="spinner-border text-primary" role="status">
  <span class="visually-hidden">Loading...</span>
</div>
<div class="spinner-border text-secondary" role="status">
  <span class="visually-hidden">Loading...</span>
</div>
<div class="spinner-border text-success" role="status">
  <span class="visually-hidden">Loading...</span>
</div>
<div class="spinner-border text-danger" role="status">
  <span class="visually-hidden">Loading...</span>
</div>
<div class="spinner-border text-warning" role="status">
  <span class="visually-hidden">Loading...</span>
</div>
<div class="spinner-border text-info" role="status">
  <span class="visually-hidden">Loading...</span>
</div>
<div class="spinner-border text-light" role="status">
```

```
<span class="visually-hidden">Loading...</span>
</div>
<div class="spinner-border text-dark" role="status">
  <span class="visually-hidden">Loading...</span>
</div>
```

Result :



Checkout official link : <https://getbootstrap.com/docs/5.3/components/spinners/#about>

19. **Bootstrap Toasts:** Push notifications to your visitors with a toast, a lightweight and easily customizable alert message.

Toasts are lightweight notifications designed to mimic the push notifications that have been popularized by mobile and desktop operating systems. They're built with flexbox, so they're easy to align and position.

Overview

Things to know when using the toast plugin:

- Toasts are opt-in for performance reasons, so **you must initialize them yourself**.
- Toasts will automatically hide if you do not specify `autohide: false`.

The animation effect of this component is dependent on the `prefers-reduced-motion` media query. See the [reduced motion section of our accessibility documentation](#).

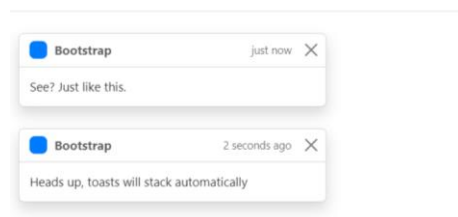
Code :

```
<div class="toast-container position-static">
  <div class="toast" role="alert" aria-live="assertive" aria-atomic="true">
    <div class="toast-header">
      
      <strong class="me-auto">Bootstrap</strong>
      <small class="text-body-secondary">just now</small>
```

```
<button type="button" class="btn-close" data-bs-dismiss="toast"
aria-label="Close"></button>
</div>
<div class="toast-body">
  See? Just like this.
</div>
</div>

<div class="toast" role="alert" aria-live="assertive" aria-
atomic="true">
  <div class="toast-header">
    
    <strong class="me-auto">Bootstrap</strong>
    <small class="text-body-secondary">2 seconds ago</small>
    <button type="button" class="btn-close" data-bs-dismiss="toast"
aria-label="Close"></button>
  </div>
  <div class="toast-body">
    Heads up, toasts will stack automatically
  </div>
</div>
</div>
```

Result :



Checkout official link :: <https://getbootstrap.com/docs/5.3/components/toasts/#overview>

20. Bootstrap Tooltip:

Documentation and examples for adding custom Bootstrap tooltips with CSS and JavaScript using CSS3 for animations and data-bs-attributes for local title storage.

Overview

Tooltips rely on the third party library Popper for positioning. You must include popper.min.js before bootstrap.js, or use one bootstrap.bundle.min.js which contains Popper.

Tooltips are opt-in for performance reasons, so you must initialize them yourself.

Tooltips with zero-length titles are never displayed.

Specify container: 'body' to avoid rendering problems in more complex components (like our input groups, button groups, etc).

Triggering tooltips on hidden elements will not work.

Tooltips for .disabled or disabled elements must be triggered on a wrapper element.

When triggered from hyperlinks that span multiple lines, tooltips will be centered. Use white-space: nowrap; on your <a>s to avoid this behavior.

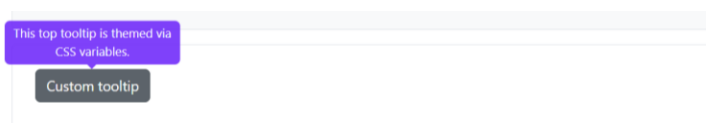
Tooltips must be hidden before their corresponding elements have been removed from the DOM.

Tooltips can be triggered thanks to an element inside a shadow DOM.

Code :

```
<style>
  .custom-tooltip {
    --bs-tooltip-bg: var(--bd-violet-bg);
    --bs-tooltip-color: var(--bs-white);
  }
</style>
<button type="button" class="btn btn-secondary"
  data-bs-toggle="tooltip" data-bs-placement="top"
  data-bs-custom-class="custom-tooltip"
  data-bs-title="This top tooltip is themed via CSS variables.">
  Custom tooltip
</button>
```

Result :



Checkout Official Link Below :

<https://getbootstrap.com/docs/5.3/components/tooltips/#overview>

21. Bootstrap Nav & Tabs:

Navigation available in Bootstrap share general markup and styles, from the base .nav class to the active and disabled states. Swap modifier classes to switch between each style.

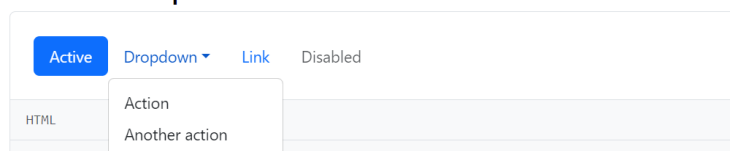
The base .nav component is built with flexbox and provide a strong foundation for building all types of navigation components. It includes some style overrides (for working with lists), some link padding for larger hit areas, and basic disabled styling.

Code:

```
<ul class="nav nav-pills">
  <li class="nav-item">
    <a class="nav-link active" aria-current="page" href="#">Active</a>
  </li>
  <li class="nav-item dropdown">
    <a class="nav-link dropdown-toggle" data-bs-toggle="dropdown" href="#"
role="button" aria-expanded="false">Dropdown</a>
    <ul class="dropdown-menu">
      <li><a class="dropdown-item" href="#">Action</a></li>
      <li><a class="dropdown-item" href="#">Another action</a></li>
      <li><a class="dropdown-item" href="#">Something else here</a></li>
      <li><hr class="dropdown-divider"></li>
      <li><a class="dropdown-item" href="#">Separated link</a></li>
    </ul>
  </li>
  <li class="nav-item">
    <a class="nav-link" href="#">Link</a>
  </li>
  <li class="nav-item">
    <a class="nav-link disabled" aria-disabled="true">Disabled</a>
  </li>
</ul>
```

Result :

Pills with dropdowns



Checkout Official Link Below: <https://getbootstrap.com/docs/5.3/components/navs-tabs/#base-nav>

22. Bootstrap NavBar :

Navbars require a wrapping `.navbar` with `.navbar-expand{-sm|-md|-lg|-xl|-xxl}` for responsive collapsing and color scheme classes.

Navbars and their contents are fluid by default. Change the container to limit their horizontal width in different ways.

Use our spacing and flex utility classes for controlling spacing and alignment within navbars. Navbars are responsive by default, but you can easily modify them to change that. Responsive behavior depends on our Collapse JavaScript plugin.

Ensure accessibility by using a `<nav>` element or, if using a more generic element such as a `<div>`, add a `role="navigation"` to every navbar to explicitly identify it as a landmark region for users of assistive technologies.

Indicate the current item by using `aria-current="page"` for the current page or `aria-current="true"` for the current item in a set.

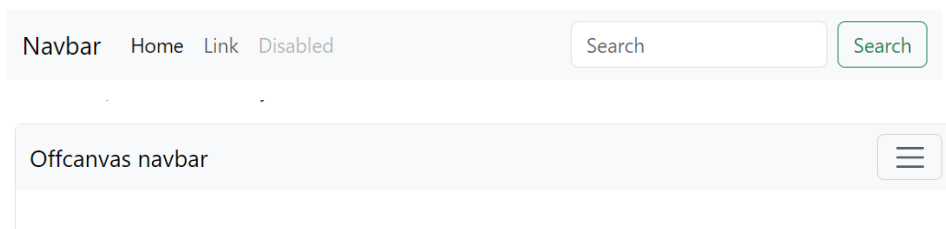
New in v5.2.0: Navbars can be themed with CSS variables that are scoped to the `.navbar` base class. `.navbar-light` has been deprecated and `.navbar-dark` has been rewritten to override CSS variables instead of adding additional styles.

Code :

```
<nav class="navbar navbar-expand-lg bg-body-tertiary">
  <div class="container-fluid">
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target="#navbarTogglerDemo03" aria-controls="navbarTogglerDemo03"
aria-expanded="false" aria-label="Toggle navigation">
      <span class="navbar-toggler-icon"></span>
    </button>
    <a class="navbar-brand" href="#">Navbar</a>
    <div class="collapse navbar-collapse" id="navbarTogglerDemo03">
      <ul class="navbar-nav me-auto mb-2 mb-lg-0">
        <li class="nav-item">
          <a class="nav-link active" aria-current="page" href="#">Home</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="#">Link</a>
        </li>
        <li class="nav-item">
          <a class="nav-link disabled" aria-disabled="true">Disabled</a>
        </li>
      </ul>
      <form class="d-flex" role="search">
```

```
<input class="form-control me-2" type="search"
placeholder="Search" aria-label="Search">
<button class="btn btn-outline-success"
type="submit">Search</button>
</form>
</div>
</div>
</nav>
```

Result :



Checkout the official website link :

<https://getbootstrap.com/docs/5.3/components/navbar/#how-it-works>

23. Bootstrap Modals :

Modals are built with HTML, CSS, and JavaScript. They're positioned over everything else in the document and remove scroll from the <body> so that modal content scrolls instead.

Clicking on the modal “**backdrop**” will automatically close the modal.

Bootstrap only supports one modal window at a time. Nested modals aren't supported as we believe them to be poor user experiences.

Modals use position: fixed, which can sometimes be a bit particular about its rendering. Whenever possible, place your modal HTML in a top-level position to avoid potential interference from other elements. You'll likely run into issues when nesting a **.modal** within another fixed element.

Once again, due to position: fixed, there are some caveats with using modals on mobile devices. See our browser support docs for details.

Due to how HTML5 defines its semantics, the autofocus HTML attribute has no effect in Bootstrap modals. To achieve the same effect, use some custom JavaScript.

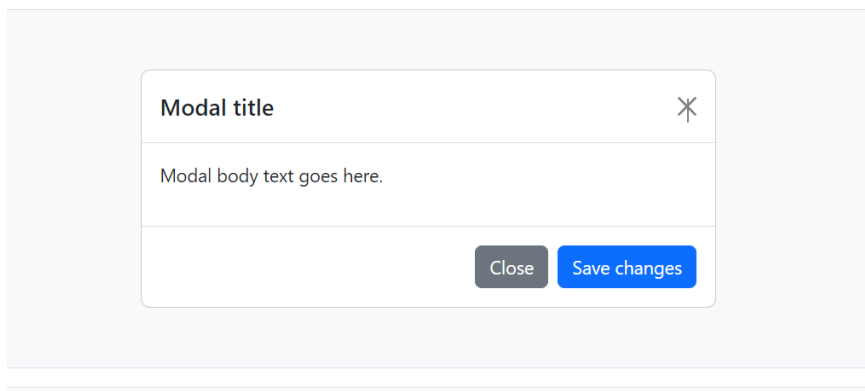
Code :

```
<script>
const myModal = document.getElementById('myModal')
const myInput = document.getElementById('myInput')
```

```
myModal.addEventListener('shown.bs.modal', () => {
  myInput.focus()
})
</script>

<body>
  <div class="modal" tabindex="-1">
    <div class="modal-dialog">
      <div class="modal-content">
        <div class="modal-header">
          <h5 class="modal-title">Modal title</h5>
          <button type="button" class="btn-close" data-bs-dismiss="modal"
aria-label="Close"></button>
        </div>
        <div class="modal-body">
          <p>Modal body text goes here.</p>
        </div>
        <div class="modal-footer">
          <button type="button" class="btn btn-secondary" data-bs-
dismiss="modal">Close</button>
          <button type="button" class="btn btn-primary">Save
changes</button>
        </div>
      </div>
    </div>
  </div>
</body>
```

Result :



Checkout the official website link :

<https://getbootstrap.com/docs/5.3/components/modal/#how-it-works>

24. Bootstrap List-Group:

List groups are a flexible and powerful component for displaying a series of content. Modify and extend them to support just about any content within.

Basic example

The most basic list group is an unordered list with list items and the proper classes. Build upon it with the options that follow, or with your own CSS as needed.

Add **.active** to a **.list-group-item** to indicate the current active selection.

Code :

```
<ul class="list-group">
  <li class="list-group-item active" aria-current="true">An active
item</li>
  <li class="list-group-item">A second item</li>
  <li class="list-group-item">A third item</li>
  <li class="list-group-item">A fourth item</li>
  <li class="list-group-item">And a fifth one</li>
</ul>
```

Result:



Checkout Official Website Link : <https://getbootstrap.com/docs/5.3/components/list-group/#basic-example>